

Medical Image Segmentation with a Vision Transformer

Segmenting the spleen from CT with Swin UNETR, and reading its attention

Project #3, chosen for two questions we wanted to answer ourselves: how a transformer does dense prediction on 3D medical images, and whether its attention explains the decision or just decorates it. We ran everything on the Shenkar GPU lab, so every constraint here is one we actually hit.

- Ori Grossman & Amit Eliya · BSc Software Engineering, Shenkar · Neural Networks, Year 3 Sem B, 2026
- Task: segment the spleen from 3D abdominal CT, then explain the call using the model's own attention
- Model: MONAI SwinUNETR, 62.2M params, MSD Task09_Spleen
- Best val Dice 94.65 — just behind published Swin UNETR 95.79 and nnU-Net 96.34, and ours is trained from scratch
- Surprise: dropping augmentation raised Dice ~3.6 points (exp2 0.9108 -> exp3 0.9465), confirmed by an epoch-matched control. Our chosen run keeps the LR schedule, drops augmentation. 4 runs, 1 wrong hypothesis, 1 ablation that settled it.

The problem, the goal, and the challenges

Automatic organ segmentation is useful, and 3D CT makes it hard

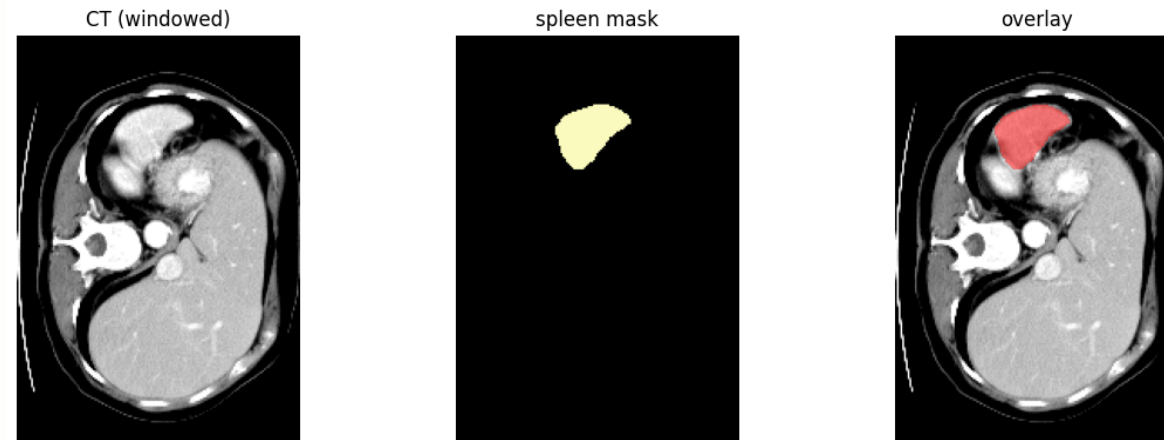
- Goal: label every voxel spleen or background on a 3D CT volume, plus show where the model looked.
- Clinically it measures organ volume and aids planning; hand-labelling every slice is slow.
- Volumetric data: a whole scan won't fit in GPU memory, so we work on 96^3 patches.
- Class imbalance: the spleen is a tiny fraction of the scan, so background dominates the loss.
- Only 41 labelled volumes total — tiny for a 62M-param transformer, and it shapes every later decision.

The data and preprocessing

MSD Task09 Spleen, 41 CT volumes, put through a standard MONAI pipeline

- MSD Task09_Spleen: 41 labelled 3D CT volumes, binary background vs spleen. Split 32 train / 9 validation.
- Picked spleen over BrainTumour, Liver and Hippocampus: small and fast, so many runs fit the budget.
- Pipeline (MONAI 1.4.0): load, channel-first, RAS orient, resample 1.5x1.5x2.0 mm, clip+normalise CT window -57..164 HU, crop foreground.
- Sampling: 4x 96³ patches per volume via RandCropByPosNegLabel (num_samples=4, batch_size=1). CacheDataset for speed.
- The -57..164 HU window is the standard abdominal soft-tissue window — drops bone and air so capacity goes to the organ.
- Downloaded via plain urllib streaming — MONAI's download_and_extract wouldn't persist on the lab's read-only NFS.

32 train / 9 val



A validation CT slice, the spleen ground-truth mask, and the two overlaid.

Related work and where we sit in it

From U-Net convolutions to transformer encoders, and our two small additions

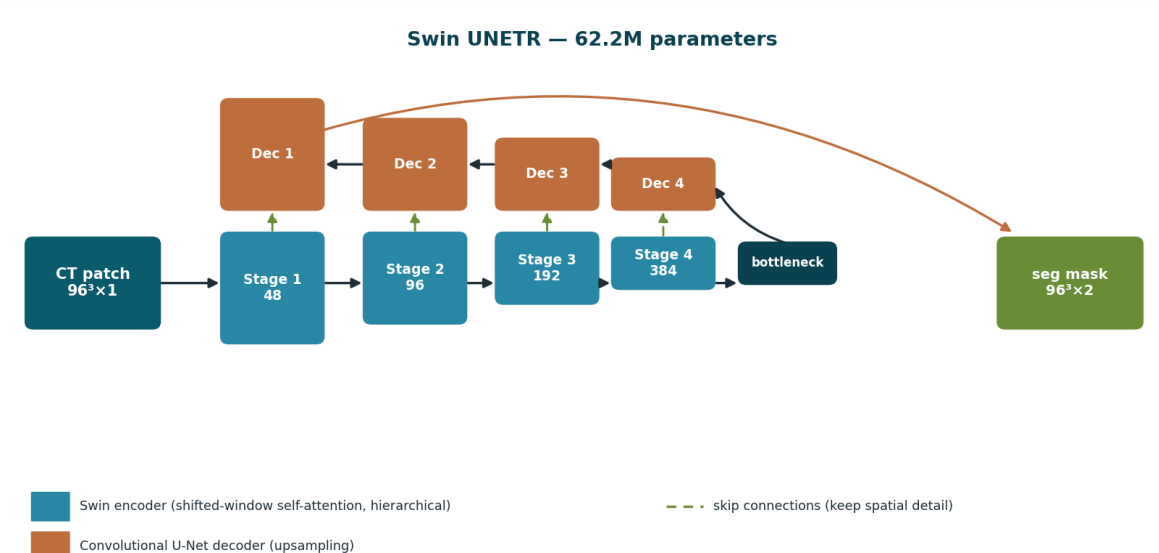
- U-Net (Ronneberger et al., 2015): encoder/decoder + skip connections. 3D U-Net (Cicek 2016) and V-Net (Milletari 2016) took it volumetric.
- nnU-Net (Isensee et al., Nature Methods 2021): self-configuring U-Net, still an extremely strong baseline — 96.34 Dice on MSD Spleen.
- ViT (Dosovitskiy et al., ICLR 2021): self-attention on image patches, but cost grows quadratically with the token count.
- Swin Transformer (Liu et al., ICCV 2021): shifted local windows for linear cost + multi-scale features. TransUNet (Chen et al., 2021) and UNETR (Hatamizadeh et al., WACV 2022) mix transformers with conv decoders.
- Swin UNETR (Hatamizadeh et al., 2022): Swin encoder + U-Net decoder, 95.79 Dice on MSD Spleen. This is the model we use.
- Our two additions: a controlled ablation showing augmentation can hurt on tiny data, and an attention-on-spleen ratio that turns the heatmap into a number.

The model: Swin UNETR, and why a transformer at all

Shifted-window self-attention in the encoder, U-Net skips in the decoder

- MONAI SwinUNETR(img_size=96, in_channels=1, out_channels=2, feature_size=48, use_checkpoint=True) — 62.2M parameters. DiceCELoss, AdamW lr=1e-4 wd=1e-5, AMP mixed precision.
- **Provenance:** MONAI's SwinUNETR implementation (Hatamizadeh et al., 2022) — we did not implement the architecture ourselves. **Trained from scratch on our 32 volumes: random initialisation, no pretrained weights.**
- Encoder: Swin transformer, shifted-window self-attention — linear token cost, hierarchical multi-scale features.
- Decoder: U-Net skips, so the transformer's global context fuses with fine local detail for a crisp mask.
- Why a transformer: self-attention lets a voxel pull evidence from far across the scan; convolutions only reach that through many stacked layers.
- Sanity forward pass before training (cheapest bug to catch): input (4,1,96,96,96) -> output (4,2,96,96,96), initial loss 1.4615, 6.0 GB of 24 GB used. Shapes matched, loss sane, headroom confirmed.

62.2M params



Swin UNETR: a transformer encoder feeds a U-Net style convolutional decoder; skip connections carry fine detail.

Training setup and the engineering constraints we hit

One L4, a VPN, and a server that dies every ~100 minutes shaped the whole plan

- Hardware: one NVIDIA L4, 24 GB VRAM (~22 usable), CUDA 13.0, reached over WireGuard VPN into Shenkar's JupyterHub GPU container.
- batch_size=2 (8 patches) OOMs once backward + the optimizer state are added, so 4 patches per step for the whole project.
- Validation: sliding_window_inference (roi 96, overlap 0.5) every 5 epochs; each run auto-saves best.pth + results.json.
- Lab session dies ~every 100 minutes. At ~1 min/epoch we capped runs at 60-100 epochs and saved often. exp2 got cut at epoch 99 (saved through 95). This timeout is the single reason no run went longer.
- A leftover forward-pass cell pinned ~11 GB of autograd graph via a hidden reference; gc and empty_cache didn't free it, only a kernel restart did. Lesson: never leave a loss or output tensor alive in a scratch cell — Python's hidden references beat gc.collect().
- nvidia-smi is missing in the container (expected), so our real GPU check was torch.cuda.is_available() returning True.

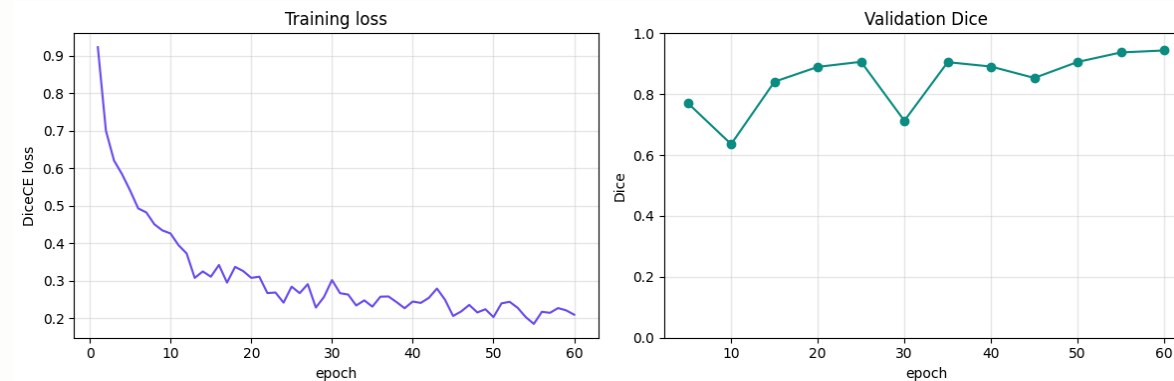
24 GB VRAM

Baseline run and what its curve told us

Fixed LR, no augmentation, good final score but an unstable ride

- Baseline: fixed LR $1e-4$, no augmentation, 60 epochs, ~61.9 minutes. Train loss fell 0.9234 -> about 0.21.
- Best validation Dice 0.9437, landing on the final epoch (60) — usually a sign the model was still improving when we stopped.
- Unstable curve: dipped to 0.6359 at epoch 10 and 0.7128 at epoch 30 before recovering.
- The dips are either genuine training instability or a 9-volume val set where one bad volume tanks the mean.
- Best-on-final plus wild dips gave our starting bet: add an LR schedule for stability and train longer.

0.9437 @ ep60



Baseline: train loss falls steadily but stays jagged, and val Dice dips to 0.64 at epoch 10 and 0.71 at epoch 30.

Our hypothesis and the hyper-parameter scan we ran

We bet on schedule plus augmentation plus more epochs, and set up four runs

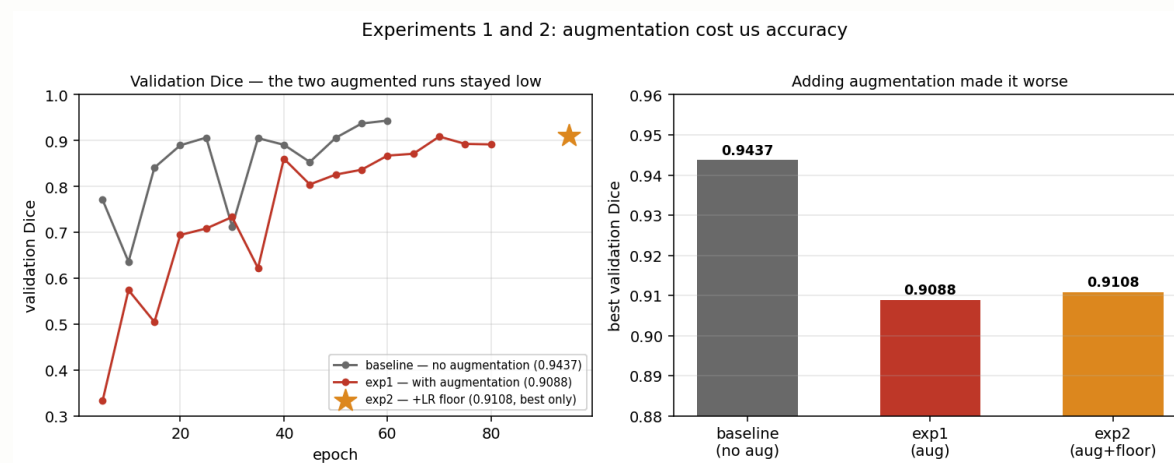
- Hypothesis: the baseline is undertrained and unstable — cosine LR + warm-up + augmentation + more epochs should raise Dice.
- Four runs, config -> best val Dice. baseline (fixed LR, no aug, 60ep) -> 0.9437.
- exp1_cosine_aug (cosine + 15ep warm-up + aug, 80ep) -> 0.9088.
- exp2_cosine_aug_floor (+ LR floor 1e-5, 100ep) -> 0.9108.
- exp3_cosine_noaug (warm-up + floor, no aug, 70ep) -> **0.9465** — the run we kept.
- 3 seeds of exp3: 0.9477 ± 0.0028 Dice. That spread is as wide as exp3's 0.003 lead over baseline, so we keep exp3 for how tightly it converges after epoch 45, not a higher score.
- Augmentation bundle: RandFlip (3 axes), RandRotate90, RandShiftIntensity.

4 runs

Experiments 1 and 2: the failure

The schedule smoothed the curve but the score got worse, and the LR floor did not save it

- exp1: cosine LR + 15-epoch warm-up + augmentation, 80 epochs. Curve smoother than baseline over its second half, but best Dice fell to 0.9088.
- That drop surprised us — everything we added is standard advice.
- First guess: undertraining, since cosine had decayed the LR to nearly zero while val Dice still looked like it was climbing at epoch 70.
- exp2 changed three things at once (warm-up 15→10, 80→100 epochs, LR floor 1e-5) — not a clean one-variable follow-up. Best Dice 0.9108, essentially flat vs exp1 (+0.002): the floor didn't rescue it, so undertraining was wrong.
- **Why exp2 has no training log:** the run was killed at epoch 99 by the lab's ~100-min timeout. It self-saved every 5 epochs, so the best result (0.9108 @ ep95) survived — we load that rather than re-train. That's why exp2 is a starred point, not a curve.



Only the runs we have discussed so far: baseline, exp1 and exp2. Both augmented runs sit below the baseline.

Diagnosis: why augmentation hurt, and our defence

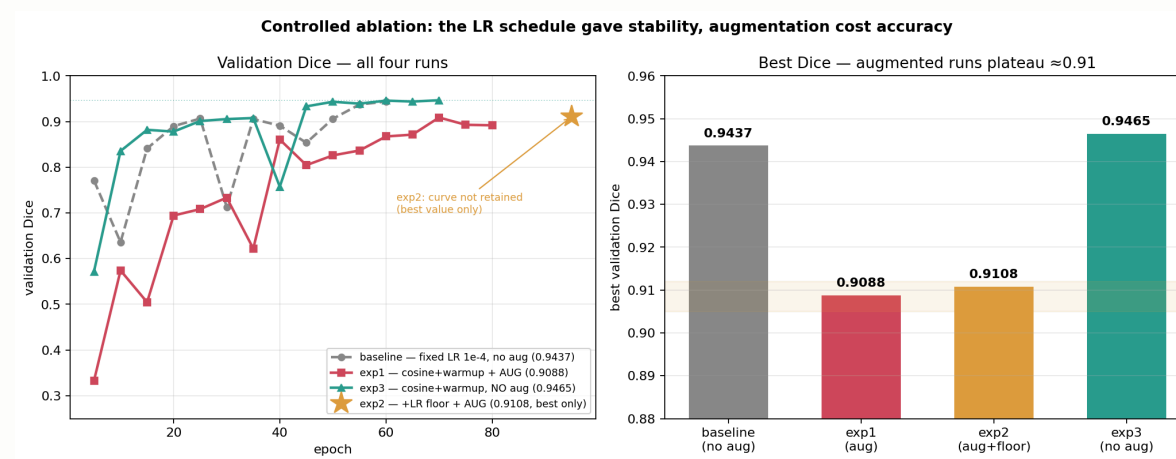
Figuring out that augmentation was the problem, then working out why, then actually testing it

- exp1 changed four variables at once (schedule, warm-up, augmentation, and 60→80 epochs) — bad hygiene, no single factor to blame.
- exp2 ruled out undertraining, leaving augmentation as the one changed variable we hadn't isolated. So exp3 holds everything else fixed and drops only augmentation.
- Why it hurts: with 32 training volumes, augmentation raises effective task difficulty and slows convergence, and a 60-to-100-epoch budget can't pay that cost back.
- Our best guess: RandRotate90 is the worst of the three — it makes body orientations that never occur at test time, so the model wastes capacity being invariant to something it won't see. We didn't test augmentations separately, so this is about the bundle, not a per-augmentation result.
- Two clean comparisons. exp3 vs baseline (both no aug) isolates the schedule — that's what smoothed the curve (baseline dipped to 0.64 and 0.71 and never settled; exp3 dips once to 0.76 at epoch 40, then holds a 0.013-wide band to the end). exp3 vs exp2 (both scheduled, floor, 10-epoch warm-up) isolates augmentation — dropping it took Dice 0.9108 -> 0.9465. exp2 ran 100 epochs and exp3 70, so we closed that gap with the epoch-matched control on the next slide. On 32 volumes, augmentation just made the task harder.
- Recipe here: schedule yes, augmentation no. exp3 is the controlled test of exactly that claim.

Experiment 3: the controlled ablation, and the run we chose

Keep the schedule, remove augmentation, and the curve finally converges

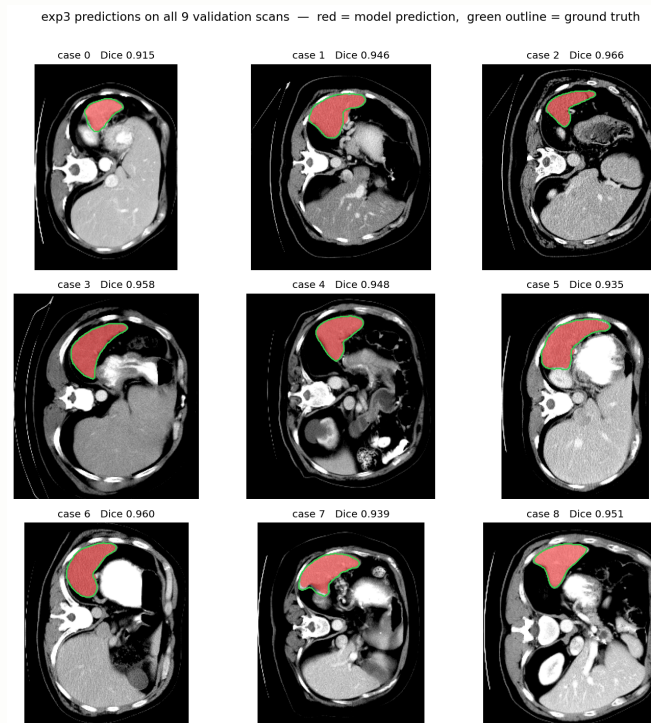
- exp3: cosine LR + 10-epoch warm-up + LR floor 1e-5, no augmentation, 70 epochs. Best validation Dice 0.9465 at epoch 70.
- That's +0.003 over baseline's 0.9437 — inside 9-volume noise (paired Wilcoxon $p = 0.20$), so we don't claim exp3 is more accurate. What it buys is convergence: exp3 dips once (0.7573 at epoch 40) then holds 0.9331–0.9465 for its last 25 epochs, while the baseline still swings 0.71–0.94 across its last 30 — a band 17× tighter.
- Ablation isolates augmentation: dropping it (exp2 0.9108 → exp3 0.9465). We then epoch-matched it — re-ran the augmented recipe at exactly 70 epochs: 0.8927 vs exp3's 0.9465, a ~5.4-pt single-variable gap. Only caveat left: augmentation went in as a bundle, so we can't blame one transform.
- Defensible recipe: keep the schedule, leave augmentation off so the model isn't fighting extra difficulty on 32 volumes. The Dice barely moved; the gain is convergence.



All four runs. The two augmented runs stay near 0.91; removing augmentation brings exp3 back to the top at 0.9465.

Results: the model finds the spleen on all 9 scans

Red = model prediction, green outline = ground truth · Dice 0.915–0.966 across the validation set



exp3 predictions on every validation patient. The prediction tracks the ground-truth boundary in all nine cases; the worst (case 0, Dice 0.915) slightly over-reaches at a low-contrast edge.

How we compare to published work

About a point behind the paper, on 32 of the 41 volumes and a single GPU

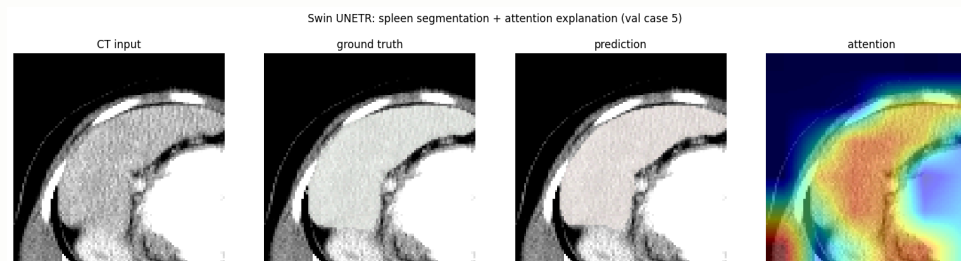
- Our best 94.65 Dice vs Swin UNETR paper 95.79 vs nnU-Net 96.34 — we're 1.14 behind the paper's model and 1.69 behind nnU-Net.
- Why they lead: 5-fold cross-validation, so every one of the 41 volumes gets trained on (we train on 32); **self-supervised pretraining** on large unlabelled CT (Swin UNETR only — nnU-Net uses none); ensembling; heavy per-task tuning; much larger compute.
- Our setup: 32 training volumes, one L4 GPU, 60-100 epochs depending on the run, no ensembling, no cross-validation, and **no pretrained weights — trained from scratch**.
- The rubric doesn't require beating them. We wanted the pipeline to land near their numbers, and 94.65 does.
- Beyond Dice (the only shared metric): HD95 3.95 mm, Surface-Dice 0.889 at 2 mm, precision 0.944, recall 0.950 — sub-4 mm boundary, balanced errors, no systematic over- or under-segmentation.
- Headline: within roughly one Dice point of a published transformer result under those limits.

94.65 vs 95.79 vs 96.34

Explainability: how we extracted the attention

Hook the deepest window-attention layer and turn its softmax into an overlay

- Forward hook on the softmax of the deepest Swin WindowAttention layer — deepest because features there are most abstract, closest to a whole-organ view. 8 WindowAttention layers found; used the last.
- Not arbitrary: we measured the on-spleen ratio for all 8 layers. Localization is a depth effect — early layers ≤ 1.24 (near 1), the last three climb to 1.7–2.2. The deep layers are where the organ-level view lives.
- Captured attention shape (1, 24, 216, 216): 1 patch, 24 heads, 216 tokens attending to 216, where $216 = 6^3$ is the window in tokens.
- Aggregation: mean over the 24 heads, then over queries $\rightarrow$ attention received per token. Reshape to $6 \times 6 \times 6$, trilinear upsample to 96^3 , overlay on the CT.
- On the analysed patch the model predicted 17,720 spleen voxels — a reasonable spleen-sized blob, a basic sanity check.
- The 4-panel figure is validation case 5: CT input, ground truth, prediction, attention.

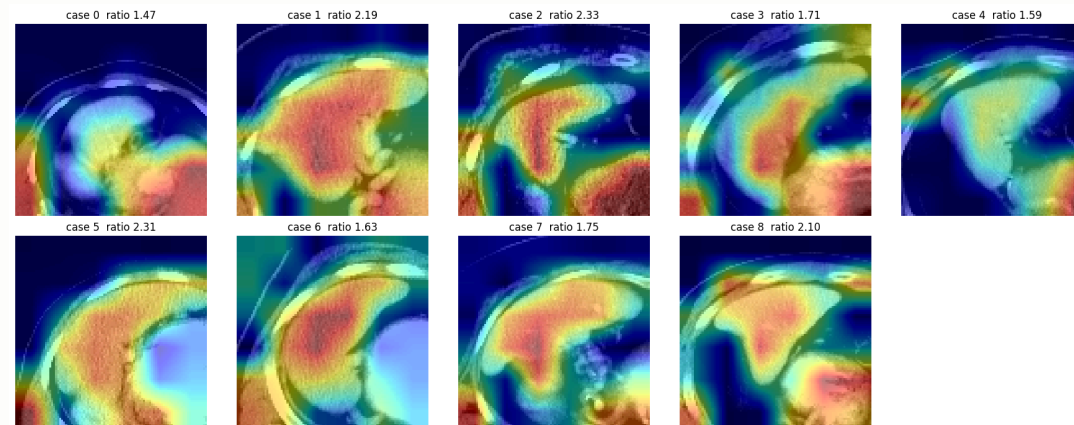


Validation case 5 (ratio 2.31): CT input, ground truth, our prediction, and the attention heatmap — a zoomed 96^3 patch, not a full scan.

Attention across all 9 validation cases

A quantified attention-on-spleen ratio, above 1 for every single case

- A metric we built for this project: attention-on-spleen ratio = mean attention inside the ground-truth spleen mask / mean attention outside it. Above 1 means the model concentrates on the organ.
- All 9 cases, not cherry-picked: 1.47, 2.19, 2.33, 1.71, 1.59, 2.31, 1.63, 1.75, 2.10 — every one above 1. Top three: case2 2.33, case5 2.31, case1 2.19 (the 4-panel figure on the previous slide is case 5).
- **Faithfulness (occlusion):** masking the highest-attention voxels drops Dice by **0.44**, versus **0.015** for a random region of the same size — paired Wilcoxon **p = 0.004**. The attention is load-bearing, not decoration.
- One number per case is what lets us claim this across the whole set, not from one nice image.



All 9 validation cases with their attention-on-spleen ratio in the title. The ratio is above 1 in every case, from 1.47 up to 2.33.

Limitations and what we would not claim

Where this falls short

- Best-at-final-epoch: baseline and the winning exp3 both peaked on their last epoch (60 and 70), likely still improving when the 100-minute server limit forced a stop.
- Tiny validation set: 9 volumes, so one hard case swings the mean a lot. No held-out test set and no k-fold (compute) — we chose the schedule and augmentation on these same 9 volumes we then report, so every Dice here is optimistic.
- Augmentation went in as a bundle: the epoch-matched control shows the bundle hurts (0.8927 vs 0.9465, both 70ep), but flip, rotate90 and shift went in together, so we can't pin it on one.
- Coarse attention: the deepest layer works on a 6^3 token grid upsampled to 96^3 , so it shows roughly where the model gathers evidence, not the crisp boundary. A sharper map needs attention roll-out.
- Attention isn't causation: a real open debate (Jain & Wallace 2019 vs Wiegrefe & Pinter 2019) — hence a testable ratio and an occlusion test, not just a heatmap.
- Lost data: exp2's per-epoch curve wasn't saved, only its best 0.9108 at epoch 95.

Conclusions and future work

What worked, the mistake we fixed, and what we'd do next

- Built a working Swin UNETR spleen-segmentation pipeline reaching 0.9465 Dice — within about a point of the published model — on 32 volumes and one GPU.
- Headline: on this tiny dataset, augmentation cost accuracy. With the schedule held fixed, dropping it took exp2 from 0.9108 to exp3's 0.9465; an epoch-matched control confirms it (the augmented recipe at 70 epochs reached only 0.8927 — a ~5.4-pt single-variable gap). It also converged: its last 25 epochs sit in a 0.013-wide band, against 0.231 for the baseline — which is why we chose exp3.
- Main lesson: change one variable at a time. exp1 changed three and cost us a whole wrong hypothesis (undertraining) before exp3 sorted it out.
- Explainability: attention-on-spleen ratio above 1 on all 9 cases (1.47–2.33), and a null control confirms it is learned — trained/spleen 1.90 vs untrained 1.11 and random-box 1.28.
- The epoch-matched control also argues against "just needs more epochs": that run peaked at epoch 40 then plateaued, so it converged and still lost ~5.4 points.
- Next: a held-out test set never touched during tuning; k-fold for error bars; a per-augmentation ablation (flip, rotate90, shift one at a time); attention roll-out for a sharper map.

0.9465, chosen for stability